Computer Organization and Architecture (CSL251)
LAB-6
2025-26 W

Switching Between Machine Mode and User Mode in RISC-V

Registers

- RISC-V architecture provides 31 user modifiable general-purpose registersy, x1 to x31.
- It also provides an additional read-only register x0, which is hard-wired to zero.

Register Name	ABI Name	Description
x0	zero	Hard-Wired Zero
x1	ra	Return Address
x2	sp	Stack Pointer
x3	gp	Global Pointer
x4	tp	Thread Pointer
x5	t0	Temporary/Alternate Link Register
x6-7	t1-t2	Temporary Register
x8	s0/fp	Saved Register (Frame Pointer)
x9	s1	Saved Register
x10-11	a0-a1	Function Argument/Return Value Registers
x12-17	a2-a7	Function Argument Registers
x18-27	s2-s11	Saved Registers
x28-31	t3-t6	Temporary Registers

Privilege modes

- To maintain security and data integrity, a system restricts how much access each process has to system resources.
- For this purpose, processes are divided into privilege modes, each with a different level of authority.

RISC-V defines three privilege levels:

- **User Mode (U) –** lowest privilege, used for normal applications
- **Supervisor Mode (S) –** medium privilege, used for operating systems
- **Machine Mode (M) –** highest privilege, full access to hardware

Privilege modes

Privilege	Value	Encoding	Abbreviation
User mode	0	00	U
Machine mode	3	11	M
Supervisor mode	1	01	S

- Each mode is represented by a value and encoding stored in CSR registers.

Control and Status Registers (CSRs) in RISC-V

- Control and Status Registers (CSRs) are special system registers in RISC-V used to control and monitor
- the processor state. Unlike general-purpose registers, CSRs manage low-level system functions such as: -
- Handling exceptions and interrupts - Switching privilege modes - Checking processor configuration -
- Enabling or disabling interrupts
- CSRs can be read, written, and in many cases, individual bits can be set or cleared.

Why CSRs are Needed

- CSRs are used when normal registers are insufficient. Typical use cases include: - Detecting why a trap
- occurred - Saving the address of a faulting instruction - Enabling timer, software, or external
- interrupts - Jumping to trap handlers
- CSRs cannot be accessed like normal registers. Instead, RISC-V provides special CSR instructions.

csrr – Read a CSR

- Copy CSR → register

Syntax

```
csrr rd, csr
```

Example

```
csrr t0, mstatus
```

- Reads mstatus into t0

Use case

- Check current privilege state
- Read trap cause
- Debug

csrw – Write a CSR

- Copy register → CSR

Syntax

csrw csr, rs1

Example

csrw mtvec, t0

- Writes t0 into mtvec

Use case

- Set trap handler
- Set return address
- Configure status bits

csrs – Set bits in a CSR

- CSR = CSR OR rs1

Syntax

```
csrs csr, rs1
```

Example

```
li t0, 0x80
csrs mie, t0
```

- Sets MTIE (bit 7) in mie

Use case

- Enable interrupts
- Set control flags safely

ecall – “Excuse me, I need help”

- A software instruction
- Used by lower privilege to request service from higher privilege
- Causes a trap immediately.

Example

- User program wants OS help → uses ecall
- CPU stops normal execution and enters Machine mode

mcause – “WHY did this happen?”

- Reason for the trap.
- Think of it as Incident report

mcause	Meaning
8	ECALL from User mode
11	ECALL from Machine mode
7	Machine timer interrupt
2	Illegal instruction

mepc – “WHERE did it happen?”

- Address of the instruction that caused the trap.
- If you don’t change mepc, returning will re-execute the same instruction → infinite loop.
- So trap handlers often do:

addi t1, t1, 4

csrw mepc, t1

- Think of mepc as bookmark

mtvec – “WHERE should I go when a trap happens?”

- Address of the trap handler.

  csrw mtvec, trap_handler

- When trap happens:

  PC ← mtvec

- Think of mtvec as emergency address

mstatus – “Global control panel”

This register controls:

- Interrupt enable
- Previous privilege mode
- Return behavior

Bit	Name	Meaning
MIE	Machine Interrupt Enable	Global interrupt ON/OFF
MPIE	Previous interrupt enable	Restored after mret
MPP	Previous privilege mode	Where to return

MPP – “Where do I go back to?”

- MPP = return ticket
- Stored inside mstatus

MPP	Mode
00	User
01	Supervisor
11	Machine

When is it used?

- During:
- mret
- CPU checks: Next mode ← MPP

mret – “Go back to where you came from”

- PC ← mepc
- Mode ← MPP
- Interrupts restored

mie – “Which bells are allowed to ring?”

- mie = bell permissions
- Controls which interrupts are enabled

Bit	Interrupt
MTIE	Timer interrupt
MSIE	Software interrupt
MEIE	External interrupt

So

- User code

ecall

Hardware does:

- Save PC → mepc
- Save reason → mcause = 8
- Save mode → MPP = U
- Jump → mtvec
- Switch to Machine mode

Then

Trap handler

- csrr t0, mcause # why
- csrr t1, mepc # where
- addi t1, t1, 4
- csrw mepc, t1
- mret

And then

After mret

- Mode ← User
- PC ← next instruction
- Program continues

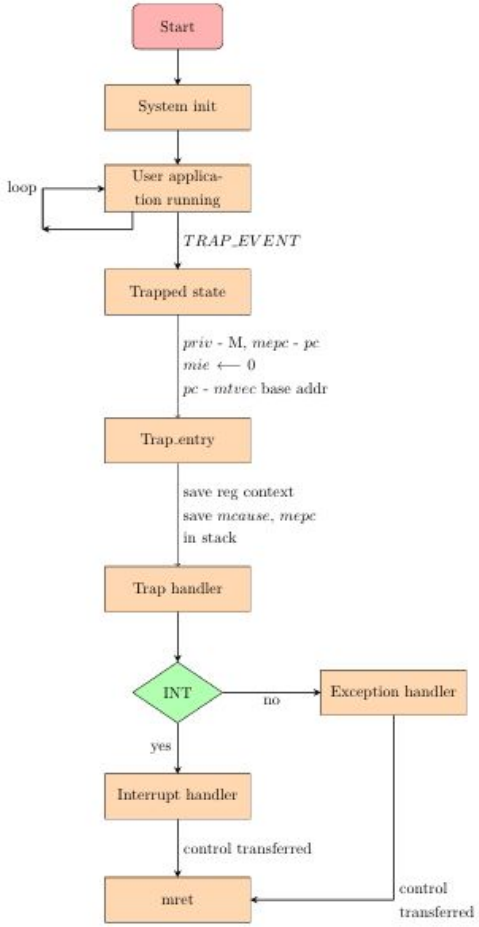


Figure 6.1: Trap occurrence and handling mechanism

Timer Interrupts

A **timer interrupt** occurs when a hardware timer reaches a predefined value and interrupts the CPU.

- It is an **asynchronous event**
- Generated by the timer hardware
- Used for:
  - Multitasking
  - Time sharing
  - Scheduling

- Handled mainly by the Operating System

mtime – “System Clock”

- Hardware timer register
- Stores current time
- Starts from 0 after power ON
- Increases automatically
- Counts in “ticks”

Think: Internal stopwatch

How mtime Works

- Works as synchronous counter
- Increases every clock tick
- Updated by hardware
- No software control
- 64-bit wide

Formula:

mtime = mtime + 1 (per tick)

mtimecmp – “Alarm Register”

- Compare register
- Stores target time
- Works with mtime
- Generates interrupt
- 64-bit register

Think: Alarm time

How Timer Interrupt is Generated

Step 1: OS sets mtimecmp

Step 2: Enable MTIE

Step 3: Hardware compares

Step 4: mtime ≥ mtimecmp

Step 5: Interrupt occurs

mcause = 7

Accessing mtime and mtimecmp

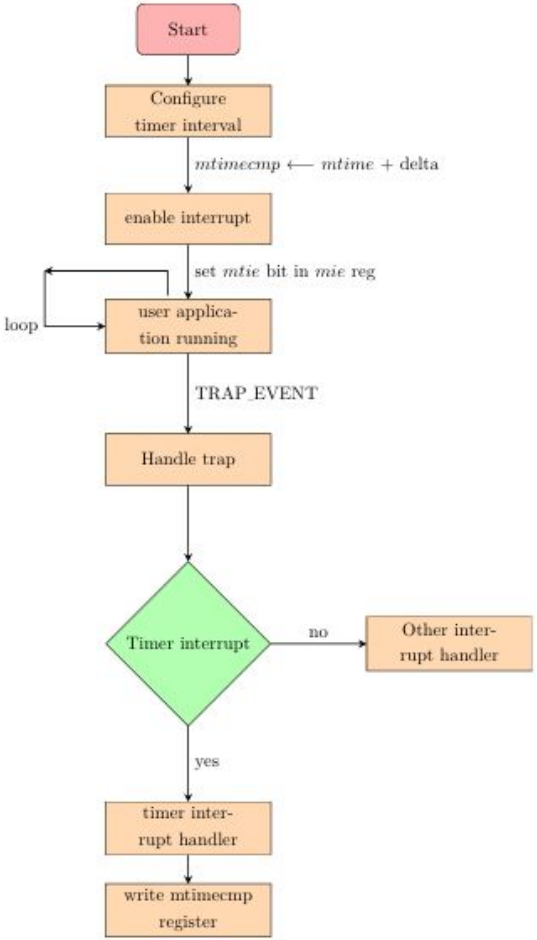
Slide Content

- Memory-mapped registers
- Access using load/store

C Example:

```
uint64_t t = *mtime;
*mtimecmp = t + 1000;
```

Assembly:
ld / sd instructions


Start
Configure timer interval
$mtimecmp \longleftarrow mtime + delta$
enable interrupt
set $mtie$ bit in mie reg
loop
user application running
TRAP_EVENT
Handle trap
Timer interrupt
no
Other interrupt handler
yes
timer interrupt handler
write mtimecmp register

Example + Comparison + Summary

Example:

mtime = 5000
Set: mtimecmp = 7000
→ Interrupt after 2000 ticks

mtime	**mtimecmp**
Time	Alarm
Read	Read/Write
Auto	Software

Summary:

- mtime = clock
- mtimecmp = alarm
- Used for scheduling